# CS4051NI/CC4059NI Fundamentals of Computing

## 70% Individual Coursework

## Final Milestone

## 2025 Spring

**Student Name: Parjun Rai**

**London Met ID: 24046801**

**College ID: NP01CP4A240222**

**Group: C8**

**Assignment Due Date: Wednesday, May 14, 2025**

**Assignment Submission Date: Wednesday, May 14, 2025**

**Word Count:**

*I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.*

# 23% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

## Match Groups

- **68** Not Cited or Quoted 22%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **4** Missing Citation 1%
  Matches that have quotation marks, but no in-text citation
- **1** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

## Top Sources

| | |
|---|---|
| 6% | Internet sources |
| 1% | Publications |
| 21% | Submitted works (Student Papers) |

## Table of Contents


1 Introduction ..... 1
1.1 Aim ..... 1
1.2 Objectives ..... 1
1.3 Tools Used ..... 2
2 Discussion & Analysis ..... 4
2.1 Algorithm ..... 4
2.2 Pseudocode ..... 5
2.2.1 Main.py ..... 5
2.2.2 Operation.py ..... 6
2.2.3 Write.py ..... 9
2.2.4 Read.py ..... 10
2.3 Flowchart ..... 12
2.4 Data Structure ..... 13
3 Program ..... 14
3.1 Displaying Products: ..... 14
3.2 Selling Products: ..... 15
3.3 Restocking Process: ..... 17
3.4 Ending Process: ..... 20
4 Testing: ..... 21
4.1 Test 1 ..... 21
4.2 Test 2: ..... 21
4.3 Test 3: ..... 22
4.4 Test 4: ..... 26
4.5 Test 5: ..... 29
5 Conclusion: ..... 34
6 Appendix: ..... 35
6.1 Main.py ..... 35
6.2 Operation.py ..... 36
6.3 Read.py ..... 41
6.4 Write.py ..... 42

## Table of Figures

Figure 1: IDLE ..... 2
Figure 2: MS-Word ..... 3
Figure 3: Draw.io ..... 3
Figure 4: Flowchart ..... 12
Figure 5: Dictionary ..... 13
Figure 6: Float ..... 13
Figure 7: Files ..... 13
Figure 8: Available Products ..... 14
Figure 9: Sell Products ..... 15
Figure 10: Selling Products ..... 16
Figure 11: Invoice.txt ..... 17
Figure 12: Invoice.txt file ..... 17
Figure 13: Restock Products ..... 17
Figure 14: Restock Products ..... 18
Figure 15: Invoice.txt ..... 19
Figure 16: Invoice.txtfile ..... 19
Figure 17: END ..... 20
Figure 18: Testing no. 1 ..... 21
Figure 19: Testing no. 2 ..... 22
Figure 20: Testing no.3 (i) ..... 24
Figure 21: Testing no.3 (ii) ..... 25
Figure 22: Testing no.3 (iii) ..... 25
Figure 23: Testing no.4 (i) ..... 27
Figure 24: Testing no.4 (ii) ..... 28
Figure 25: Testing no.4 (iii) ..... 28
Figure 26: Testing no.5 (i) ..... 31
Figure 27: Testing no.5 (i) ..... 31
Figure 28: Testing no.5 (i) ..... 32
Figure 29: Testing no.5 (ii) ..... 33
Figure 30: Testing no.5 (ii) ..... 33

# 1 Introduction

The primary goal of the assignment is to build a useful program based on a real-life situation. The system helps WeCare, a local shop that sells beauty and skincare products, manage its sales and stock. It can show available products, apply a “Buy 3, Get 1 Free” offer, create invoices for both customers and suppliers, keep track of product stock, and save everything using files.

## 1.1 Aim

This coursework's goal is to provide us with a clear sense of direction while we work on the project. Our work is guided by objectives, which also help us stay focused, make efficient use of our time and resources, and assess our progress. One major objective is to enhance our communication, programming, and documentation abilities, all of which are critical for our future employment. We also learned how to use Python to tackle practical problems.

## 1.2 Objectives

- Design a simple system to help WeCare manage their beauty and skincare product inventory easily.
- Automatically apply the "Buy 3 Get 1 Free" offer during sales to improve customer experience.
- Generate invoices for both customers and suppliers after every transaction.
- Update and save inventory data using files, so product information is not lost between sessions.

## 1.3 Tools Used

- **IDLE:**

  IDLE is a built-in development environment that comes with Python, offering a simple and intuitive interface for writing, testing, and running Python code. It's especially useful for beginners because it provides helpful features like color-coded syntax, automatic indentation, and an interactive shell for quick testing. We used IDLE throughout this project for its ease of use and ability to quickly debug and experiment with our code in real-time.

*Figure 1: IDLE*

- **Ms-Word:**

  Microsoft Word is a versatile and powerful word processing program developed by Microsoft, designed to make it easy for users to create, edit, format, and share text documents. As part of the Microsoft Office suite, it has become the standard tool for creating documents in both personal and professional settings. Whether you're working on resumes, letters, reports, essays, invoices, or brochures, Microsoft Word offers a range of features that help make the process simple and efficient. It includes extensive formatting options, the ability to insert images, adjust page layouts, and even collaborate with others on the same document. These features make it the go-to program for most document-related tasks.

*Figure 2: MS-Word*

- **Draw.io:**

  Draw.io is a free, web-based tool for creating all kinds of diagrams, including organization charts, flowcharts, mind maps, and more. It works directly with Google Drive, so you can automatically save your work to your Google Workspace or Gmail account. As an open-source and self-hosted application, Draw.io lets you easily build diagrams using a drag-and-drop interface with pre-designed templates. It also allows you to import and export diagrams in different formats, making it a flexible and powerful tool for teams working on visual projects.

*Figure 3: Draw.io*

# 2 Discussion & Analysis

## 2.1 Algorithm

Step1: Start the program.

Step 2: Show the user the transaction menu.

Step 3: Ask the user to pick an option by typing a number.

Step 4: Check that the input is a valid number.

Step 5: If the user picks option 1:

- Show them the list of products available.

Step 6: If the user picks option 2:

- Ask them which product they want to sell and how much it
- If it's in stock, confirm the product and quantity.
- Calculate the total price and update the inventory.
- Create an invoice with the customer's details, product, quantity, and total cost.

Step 7: If the user picks option 3:

- Ask them which product they want to restock and how much they need
- Confirm the restock details.
- Calculate the cost to restock and update the inventory.
- Create a restock invoice with details like product, quantity, and total cost.

Step 8: If the user picks option 4:

- Exit the transaction menu and close the program.

## 2.2 Pseudocode

### 2.2.1 Main.py

```
BEGIN PROGRAM

 IMPORT the following:
   - read_file from the module 'read'
   - show_products, sell_product, restock_product from the module 'operation'

 DEFINE the main() function:

    CALL read_file() to load the product data from a file
      STORE the result in a variable called 'products'

    LOOP infinitely:
          DISPLAY a formatted menu with options:
            - 1: Display Products
            - 2: Sell Products
            - 3: Restock Products
            - 4: Exit

         PROMPT the user to select an option (1-4)
            STORE the user input in a variable named 'option'

        PERFORM actions based on the value of 'option':
            - IF option is '1':
                CALL show_products(products)
            - ELSE IF option is '2':
                CALL sell_product(products)
            - ELSE IF option is '3':
                CALL restock_product(products)
```

```
        - ELSE IF option is '4':
            PRINT a goodbye message
            EXIT the loop
        - ELSE:
            PRINT "Invalid option!"

 CHECK if this file is the main program being executed:
   - IF YES, CALL the main() function

END PROGRAM
```

### 2.2.2 Operation.py

```
FUNCTION show_products(products):

    DISPLAY heading "Available Products"
    DISPLAY column titles: Product, Brand, Price, In Stock, From
    DISPLAY a separator line

    FOR each product 'p' in products:
        FORMAT product details into a line:
            - name (left-justified)
            - brand (left-justified)
            - price (rounded to 2 decimal places)
            - stock quantity
            - country of origin
        DISPLAY the formatted line

END FUNCTION

FUNCTION sell_product(products):
```

```
CALL show_products(products)
PROMPT user to enter customer name
PROMPT user to enter product name
INITIALIZE variable 'found' as None

FOR each product in products:
    IF product name matches user input (case-insensitive):
        SET found to that product
        BREAK the loop

IF found is still None:
    DISPLAY "Product not found"
    EXIT function

TRY:
    PROMPT user to enter quantity
    IF quantity <= 0:
        DISPLAY "Please enter positive number"
        EXIT function

    IF quantity > stock in 'found':
        DISPLAY message with available stock
        EXIT function

    CALCULATE free items = quantity // 3
    CALCULATE total = quantity * price
    REDUCE found's stock by (quantity + free)

    DISPLAY formatted purchase receipt:
        - customer name, product, brand
        - price, quantity + free, total amount
```

```
            - thank-you message

        CALL save_products(products)

        CREATE invoice text with:
            - customer, product, brand, price, quantity (+free), total
        CALL save_invoice(invoice text)

    EXCEPT ValueError:
        DISPLAY "Please enter a valid number"

END FUNCTION

FUNCTION restock_product(products):

    CALL show_products(products)

    TRY:
        PROMPT user to enter staff name
        PROMPT user to enter product name
        PROMPT user to enter quantity to add
        PROMPT user to enter new cost (optional)

        FOR each product in products:
            IF product name matches input:
                INCREASE product stock by quantity

                IF new cost is provided:
                    UPDATE product cost
                    RECALCULATE price = cost * 2 (rounded)
```

```
            CALL save_products(products)

            CREATE invoice text with:
                - staff name, product, brand, added quantity, new cost
            CALL save_invoice(invoice text)

            DISPLAY formatted restock confirmation receipt

            EXIT function

        DISPLAY "Product not found"

    EXCEPT ValueError:
        DISPLAY "Invalid input!"

END FUNCTION
```

### 2.2.3 Write.py

```
FUNCTION save_products(products, filename = "products.txt"):

    TRY:
        OPEN the file with the given filename in write mode

        FOR each product 'p' in the products list:
            CONVERT product data to a comma-separated string:
                name, brand, stock, cost, country
            WRITE this string followed by a newline to the file

    EXCEPT (if an error occurs):
        DISPLAY "Error saving products file"
```

```
END FUNCTION

FUNCTION save_invoice(transaction_details, invoice_filename = "invoice.txt"):

    TRY:
        OPEN the file with the given invoice_filename in append mode
        WRITE the transaction_details followed by two newlines

    EXCEPT (if an error occurs):
        DISPLAY "Error saving invoice file"

END FUNCTION
```

### 2.2.4 Read.py

```
FUNCTION read_file(optional parameter filename = "products.txt"):

    INITIALIZE an empty list called products

    TRY to do the following:
        OPEN the file with the given filename in read mode

        FOR each line in the file:
            IF the line is not empty after removing whitespace:
                SPLIT the line by commas into parts
                EXTRACT the following values from the parts:
                    name = first part (trimmed)
                    brand = second part (trimmed)
                    qty = third part converted to integer
                    cost = fourth part converted to float
                    country = fifth part (trimmed)

                CALCULATE price as cost multiplied by 2, rounded to 2 decimal places
```

```
            CREATE a product dictionary with:
                - name
                - brand
                - stock (qty)
                - cost
                - country
                - price

            ADD the product dictionary to the products list

        RETURN the list of products

    IF any error occurs:
        PRINT "Something went wrong while reading the file."
        RETURN an empty list

END FUNCTION
```

## 2.3 Flowchart

*Figure 4: Flowchart*

## 2.4 Data Structure

- **Dictionary:** A dictionary in Python is a simple way to store information in pairs, like a label and its value. It keeps things in order, lets you change the data, and doesn't allow the same label (or key) twice. It's great for storing details about things like products. For example, you can use a dictionary to store a product's name, brand, and price.

```
product = {
    'name': name,
    'brand': brand,
    'stock': qty,
    'cost': cost,
    'country': country,
    'price': round(cost * 2, 2)
}
```

*Figure 5: Dictionary*

- **Float:** A float (short for floating-point number) is a data type in Python that represents real numbers (numbers with decimals)

```
free = qty // 3
total = qty * found['price']
found['stock'] -= (qty + free)

# Receipt
print("\n" + "—" * 40)
```

*Figure 6: Float*

- **Files:** Files Object are used to read write and store data for several purposes.

```
def save_products(products, filename="products.txt"):
    """Save products to file."""
    try:
        with open(filename, 'w') as file:
            for p in products:
                file.write(p['name'] + "," + p['brand'] + "," + str(p['stock']) + "," + str(p['cost']) + "," + p['country'] + "\n")
    except:
```

*Figure 7: Files*

# 3 Program

Whole execution of the program as Below:

## 3.1 Displaying Products:

```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
          SkinCare Inventory System:
_____________________________________________________________
          1. Display Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
          2. Sell Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
          3. Restock Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
          4. Exit
_____________________________________________________________
Choose option (1-4): 1

=== Available Products ===
Product              Brand          Price     In Stock  From
------------------------------------------------------------
Vitamin C Serum     Garnier        $2000.0    10      France
Skin Cleanser       Cetaphil       $560.0     10      Switzerland
Sunscreen           Aqualogica     $2800.0    11      India
```

*Figure 8: Available Products*

- When the program is executing the following Transaction Menu will pop up like above figure. The user must choose option 1 to see the Product details such as brand etc.

## 3.2 Selling Products:

```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
            SkinCare Inventory System:
_________________________________________________________________
            1. Display Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
            2. Sell Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
            3. Restock Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
            4. Exit
_________________________________________________________________
Choose option (1-4): 2
```

*Figure 9: Sell Products*

- The user must input number 2 to sell the products.

```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
            SkinCare Inventory System:

_____________________________________________________________
            1. Display Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
            2. Sell Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
            3. Restock Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
            4. Exit

_____________________________________________________________
Choose option (1-4): 2

=== Available Products ===
Product              Brand          Price     In Stock  From
------------------------------------------------------------
Vitamin C Serum     Garnier        $2000.0    10       France
Skin Cleanser       Cetaphil       $560.0     10       Switzerland
Sunscreen           Aqualogica     $2800.0    7        India
Enter customer name: Parjun
Enter product name: Skin Cleanser
Quantity: 3

=========================================
             WECARE SKINCARE
=========================================
 Customer:  Parjun
 Product:   Skin Cleanser
 Brand:     Cetaphil
_________________________________________
 Price:     $560.0
 Quantity:  3 (+1 free)
_________________________________________
 TOTAL:     $1680.0
=========================================
      Thank you for your purchase!
=========================================
```

*Figure 10: Selling Products*

- After the user inputs number 2 the program will show the stocks of the products then the user must input their name, Product name, and Quantity in order to sell the products then the invoice will be generated like in the above figure.

| | | | |
|---|---|---|---|
| invoice | 5/14/2025 3:04 AM | Text Document | 1 KB |
| main | 5/14/2025 2:13 AM | Python File | 2 KB |
| operation | 5/13/2025 10:38 PM | Python File | 5 KB |
| products | 5/14/2025 3:04 AM | Text Document | 1 KB |

*Figure 11: Invoice.txt*

- Then the invoice will be generated in txt file like in the above diagram.

```
Sell Products
Customer: Parjun
Product: Skin Cleanser
Brand: Cetaphil
Price: $560.0
Quantity: 3 (+1 free)
Total: $1680.0
```

*Figure 12: Invoice.txt file*

- Now the invoice will be generated in the .txt file like the above diagram.

## 3.3 Restocking Process:

```
*****************************************************************************

++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
          SkinCare Inventory System:
____________________________________________________________
          1. Display Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
          2. Sell Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
          3. Restock Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
          4. Exit
____________________________________________________________
Choose option (1-4): 3
```

*Figure 13: Restock Products*

- The user must input number 3 to Restock the products.

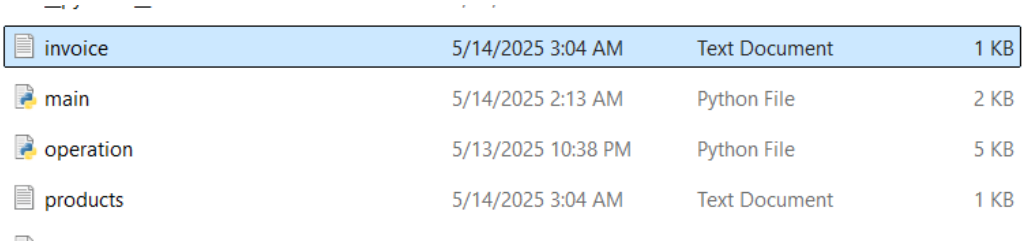
```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
           SkinCare Inventory System:
_____________________________________________________________
           1. Display Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           2. Sell Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           3. Restock Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           4. Exit
_____________________________________________________________
Choose option (1-4): 3

=== Available Products ===
Product              Brand          Price     In Stock  From
-------------------------------------------------------------
Vitamin C Serum     Garnier        $2000.0    10       France
Skin Cleanser       Cetaphil       $560.0     6        Switzerland
Sunscreen           Aqualogica     $2800.0    7        India
Enter staff name: Parjun
Product name: Skin Cleanser
Quantity to add: 4
New cost (press Enter to keep current):

=========================================
          RESTOCK CONFIRMATION
=========================================
 Staff:      Parjun
 Product:    Skin Cleanser
 Brand:      Cetaphil
-----------------------------------------
 Added Qty: 4
 New Cost:  $280.0
=========================================
    Inventory updated successfully!
=========================================
```

*Figure 14: Restock Products*

- After the user inputs number 3 the program will show the stocks of the products then the user must input their name, Product name, and Quantity in order to restock the products then the invoice will be generated like in the above figure.



| | | | |
|---|---|---|---|
| invoice | 5/14/2025 3:04 AM | Text Document | 1 KB |
| main | 5/14/2025 2:13 AM | Python File | 2 KB |
| operation | 5/13/2025 10:38 PM | Python File | 5 KB |
| products | 5/14/2025 3:04 AM | Text Document | 1 KB |

*Figure 15: Invoice.txt*

- Then the invoice will be generated in txt file like in the above diagram.

```
Restock Products
Staff: Parjun
Product: Skin Cleanser
Brand: Cetaphil
Added Quantity: 4
New Cost: $280.0
```

*Figure 16: Invoice.txtfile*

- Now the invoice will be generated in the .txt file like the above diagram.

## 3.4 Ending Process:

To Terminate the program user must choose option 4 then it will end the program.

```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
            SkinCare Inventory System:
_________________________________________________________________
            1. Display Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
            2. Sell Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
            3. Restock Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
            4. Exit
_________________________________________________________________
Choose option (1-4): 4
Goodbye!
```

*Figure 17: END*

# 4 Testing:

## 4.1 Test 1

| | | |
|---|---|---|
| 1 | Objective | To test the implementation of try, except. |
| 2 | Action | The system is executed.<br>Input the invalid values. |
| 3 | Expected Result | An invalid input message should appear on the screen. |
| 4 | Actual Result | An invalid input message popped up. |
| 5 | Conclusion | Testing success. |

```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
            SkinCare Inventory System:
_________________________________________________________________
            1. Display Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
            2. Sell Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
            3. Restock Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
            4. Exit
_________________________________________________________________
Choose option (1-4): 5
Invalid option!
```

*Figure 18: Testing no. 1*

## 4.2 Test 2:

| | | |
|---|---|---|
| 1 | Objective | To check if the system Selection purchase and sale of products<br>Provide the negative value as input<br>Provide the non existed value as input |
| 2 | Action | • The system is executed.<br>• Option 1: Product details are displayed. Option 2: was selected which was sell product.<br>• A negative value was given as input. |

| | | |
|---|---|---|
| | | • A non-existent value was given as input. |
| 3 | Expected Result | An invalid Input message would appear . |
| 4 | Actual Result | An invalid input message appeared. |
| 5 | Conclusion | Test Success. |

```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
            SkinCare Inventory System:
_____________________________________________________________
            1. Display Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
            2. Sell Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
            3. Restock Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
            4. Exit
_____________________________________________________________
Choose option (1-4): 2

=== Available Products ===
Product              Brand          Price      In Stock  From
-------------------------------------------------------------
Vitamin C Serum      Garnier        $2000.0    10        France
Skin Cleanser        Cetaphil       $560.0     10        Switzerland
Sunscreen            Aqualogica     $2800.0    7         India
Enter customer name: Parjjun
Enter product name: slsl
Product not found!
```

*Figure 19: Testing no. 2*

## 4.3 Test 3:

| | | |
|---|---|---|
| 1 | Objective | To check if File generation of purchase of product(s) (Purchasing multiple product(s)) |
| 2 | Action | • The program is executed.<br>• Option 1 (dispalying the available product details).<br>• Option 3 (purchasing the product available in stock) was selected.<br>• The values were given as: |

| | | |
|---|---|---|
| | | Enter the product name: Skin Cleanser<br><br>Enter the quantity you want to add: 4<br><br>Enter the name: Parjun |
| 3 | Expected Result | The restock product Invoice details would be generated in a .txt file |
| | | |
| 4 | Actual Result | The purchasing(restock the product)Invoice details was generated in a .txt file |
| 5 | Conclusion | Testing success. |

```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
           SkinCare Inventory System:
_____________________________________________________________
           1. Display Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           2. Sell Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           3. Restock Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           4. Exit
_____________________________________________________________
Choose option (1-4): 3

=== Available Products ===
Product              Brand          Price     In Stock  From
------------------------------------------------------------
Vitamin C Serum     Garnier        $2000.0    10       France
Skin Cleanser       Cetaphil       $560.0     6        Switzerland
Sunscreen           Aqualogica     $2800.0    7        India
Enter staff name: Parjun
Product name: Skin Cleanser
Quantity to add: 4
New cost (press Enter to keep current):

==========================================
          RESTOCK CONFIRMATION
==========================================
 Staff:     Parjun
 Product:   Skin Cleanser
 Brand:     Cetaphil
------------------------------------------
 Added Qty: 4
 New Cost:  $280.0
==========================================
    Inventory updated successfully!
==========================================
```

*Figure 20: Testing no.3 (i)*

```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
           SkinCare Inventory System:
_____________________________________________________________
           1. Display Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           2. Sell Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           3. Restock Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           4. Exit
_____________________________________________________________
Choose option (1-4): 3

=== Available Products ===
Product              Brand          Price     In Stock  From
-------------------------------------------------------------
Vitamin C Serum     Garnier        $2000.0    10       France
Skin Cleanser       Cetaphil       $560.0     6        Switzerland
Sunscreen           Aqualogica     $2800.0    7        India
Enter staff name: Parjun
Product name: Skin Cleanser
Quantity to add: 4
New cost (press Enter to keep current):

=========================================
          RESTOCK CONFIRMATION
=========================================
 Staff:     Parjun
 Product:   Skin Cleanser
 Brand:     Cetaphil
-----------------------------------------
 Added Qty: 4
 New Cost:  $280.0
=========================================
    Inventory updated successfully!
=========================================
```

*Figure 21: Testing no.3 (ii)*

```
Restock Products
Staff: Parjun
Product: Skin Cleanser
Brand: Cetaphil
Added Quantity: 4
New Cost: $280.0
```

*Figure 22: Testing no.3 (iii)*

## 4.4 Test 4:

| | | |
|---|---|---|
| 1 | Objective | File generation of sales process of products (Selling multiple products. To check if the Show complete sales process<br>Show output in the shell as well<br>Finally show the purchased products details in a text file |
| 2 | Action | • The program executed.<br>• Option 1 (displays the product details).<br>• Option 2 (sell products).<br>• The values were given as:<br><br>Enter the product name: Skin Cleanser<br><br>Enter the customer's name: Parjun<br><br>Enter the quantity: 3 |
| 3 | Expected Result | Invoice details would be generated in a .txt file which was sold skin cleanser. |
| 4 | Actual Result | The sold product Invoice details was generated in a .txt file |
| 5 | Conclusion | Testing success. |

```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
           SkinCare Inventory System:
____________________________________________________________
           1. Display Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           2. Sell Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           3. Restock Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           4. Exit
____________________________________________________________
Choose option (1-4): 2

=== Available Products ===
Product              Brand          Price     In Stock  From
------------------------------------------------------------
Vitamin C Serum     Garnier        $2000.0    10       France
Skin Cleanser       Cetaphil       $560.0     10       Switzerland
Sunscreen           Aqualogica     $2800.0    7        India
Enter customer name: Parjun
Enter product name: Skin Cleanser
Quantity: 3

==========================================
             WECARE SKINCARE
==========================================
 Customer:  Parjun
 Product:   Skin Cleanser
 Brand:     Cetaphil
------------------------------------------
 Price:     $560.0
 Quantity:  3 (+1 free)
------------------------------------------
 TOTAL:     $1680.0
==========================================
      Thank you for your purchase!
==========================================
```

*Figure 23: Testing no.4 (i)*

```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
           SkinCare Inventory System:
_____________________________________________________________
           1. Display Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           2. Sell Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           3. Restock Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           4. Exit
_____________________________________________________________
Choose option (1-4): 2

=== Available Products ===
Product              Brand          Price     In Stock  From
------------------------------------------------------------
Vitamin C Serum      Garnier        $2000.0   10        France
Skin Cleanser        Cetaphil       $560.0    10        Switzerland
Sunscreen            Aqualogica     $2800.0   7         India
Enter customer name: Parjun
Enter product name: Skin Cleanser
Quantity: 3

==========================================
             WECARE SKINCARE
==========================================
 Customer:  Parjun
 Product:   Skin Cleanser
 Brand:     Cetaphil
------------------------------------------
 Price:     $560.0
 Quantity:  3 (+1 free)
------------------------------------------
 TOTAL:     $1680.0
==========================================
      Thank you for your purchase!
==========================================
```

*Figure 24: Testing no.4 (ii)*

```
Sell Products
Customer: Parjun
Product: Skin Cleanser
Brand: Cetaphil
Price: $560.0
Quantity: 3 (+1 free)
Total: $1680.0
```

*Figure 25: Testing no.4 (iii)*

## 4.5 Test 5:

| | | |
|---|---|---|
| 1 | Objective | Show the update in stock of product(s)<br>Show the quantity being deducted while purchasing the product (Update should be reflected in a .txt file as well)<br>Show the quantity being added while selling the product (Update should be reflected in a .txt file as well) |
| 2 | Action | • The Program executed.<br>• Enter the name prompt was shown.<br>• Option 1 (Showing the product details).<br>• Option 2 (Selling product) was selected.<br>• The values were given as:<br>• Option 2 (sell products) was selected.<br>• The values were:<br><br>Enter the product name: Skin Cleanser<br><br>Enter the customer's name: Parjun<br><br>• Enter the quantity: 3<br><br>Restocking process<br>The Program executed.<br><br>Transaction menu:<br><br>• Option 1 (Showing the available product details).<br>• Option 3 (purchasing the product) was selected.<br>• The values were given as:<br><br>Enter the product name: Skin Cleanser |

| | | |
|---|---|---|
| | | Enter the quantity you want to add: 4<br><br>Enter the name: Parjun |
| 3 | Expected Result | The availability of the product should change in the updates should be reflected accurately in the corresponding .txt file. |
| 4 | Actual Result | The availability of the product should change in the updates should be reflected accurately in the corresponding .txt file. |
| 5 | Conclusion | Testing success. |

```
Vitamin C Serum,Garnier,10,1000.0,France
Skin Cleanser,Cetaphil,10,280.0,Switzerland
Sunscreen,Aqualogica,10,1400.0,India
```

*Figure 26: Testing no.5 (i)*

```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
           SkinCare Inventory System:
_________________________________________________________________
           1. Display Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           2. Sell Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           3. Restock Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           4. Exit
_________________________________________________________________
Choose option (1-4): 2

=== Available Products ===
Product              Brand          Price     In Stock  From
------------------------------------------------------------
Vitamin C Serum     Garnier        $2000.0    10       France
Skin Cleanser       Cetaphil       $560.0     10       Switzerland
Sunscreen           Aqualogica     $2800.0    10       India
Enter customer name: Parjun
Enter product name: Skin Cleanser
Quantity: 3

=========================================
             WECARE SKINCARE
=========================================
 Customer:  Parjun
 Product:   Skin Cleanser
 Brand:     Cetaphil
-----------------------------------------
 Price:     $560.0
 Quantity:  3 (+1 free)
-----------------------------------------
 TOTAL:     $1680.0
=========================================
      Thank you for your purchase!
=========================================
```

*Figure 27: Testing no.5 (i)*

```
Vitamin C Serum,Garnier,10,1000.0,France
Skin Cleanser,Cetaphil,6,280.0,Switzerland
Sunscreen,Aqualogica,10,1400.0,India
```

*Figure 28: Testing no.5 (i)*

```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
           SkinCare Inventory System:
_____________________________________________________________
           1. Display Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           2. Sell Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           3. Restock Products
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           4. Exit
_____________________________________________________________
Choose option (1-4): 3

=== Available Products ===
Product              Brand          Price     In Stock  From
-------------------------------------------------------------
Vitamin C Serum      Garnier        $2000.0    10       France
Skin Cleanser        Cetaphil       $560.0     6        Switzerland
Sunscreen            Aqualogica     $2800.0    10       India
Enter staff name: Parjun
Product name: Skin Cleanser
Quantity to add: 4
New cost (press Enter to keep current):

=========================================
          RESTOCK CONFIRMATION
=========================================
 Staff:      Parjun
 Product:    Skin Cleanser
 Brand:      Cetaphil
-----------------------------------------
 Added Qty: 4
 New Cost:  $280.0
=========================================
    Inventory updated successfully!
=========================================
```

*Figure 29: Testing no.5 (ii)*

```
Vitamin C Serum,Garnier,10,1000.0,France
Skin Cleanser,Cetaphil,10,280.0,Switzerland
Sunscreen,Aqualogica,10,1400.0,India
```

*Figure 30: Testing no.5 (ii)*

# 5 Conclusion:

In Conclusion, in this report, we have provided an overview of our beauty and skincare inventory system that we designed to facilitate product sale and restocking. Through Python scripts, we have shown the major functionalities of the system, such as showing available products, selling products to customers with a discount offer, restocking products with new prices, and printing customer and supplier invoices to track the transactions appropriately.

# 6 Appendix:

## 6.1 Main.py

```
# main.py
# Importing functions from other Python files (called modules)
from read import read_file# This reads product data from a text file
from operation import show_products, sell_product, restock_product# These handle
displaying, selling, and restocking products

# This is the main function that runs the program
def main():
    """Main menu loop."""
    products = read_file() # Load the list of products from the file

 # Start an infinite loop to keep showing the menu until the user wants to exit
    while True:
        # Show the main menu options

print("\n*******************************************************************************")

print("\n+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
+  ")
        print("          SkinCare Inventory System:                        ")

print("_____________________________________________________________________")
        print("          1. Display Products                               ")

print("~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~")
        print("          2. Sell Products                                  ")
```

```
print("~~~
```

```
from write import save_products, save_invoice # Import functions to save product data
and invoices to files

# Function to show the list of available products
def show_products(products):
    """Display available products."""
    print("\n=== Available Products ===")
    print("Product" + " " * 14 + "Brand" + " " * 10 + "Price" + " " * 5 + "In Stock" + "  " +
"From")
    print("-" * 60)
    # Go through each product in the list and print its details in a formatted way
    for p in products:
    for p in products:
        line = p['name'].ljust(20)
        line += p['brand'].ljust(15)
        line += "$" + str(round(p['price'], 2)).ljust(10)
        line += str(p['stock']).ljust(8)
        line += p['country']
        print(line)

def sell_product(products):# Function to handle selling a product
    """Handle product sale and receipt generation."""
    show_products(products)
    customer_name = input("Enter customer name: ").strip()
    product_name = input("Enter product name: ").strip().lower()
    found = None# This will hold the product if we find it

 # Search for the product in the list
    for p in products:
        if p['name'].lower() == product_name:
            found = p
```

```
            break

    if not found:
        print("Product not found!")
        return

    try:
        qty = int(input("Quantity: "))# Ask how many items the customer wants
        if qty <= 0:
            print("Please enter positive number")
            return

        if qty > found['stock']:
            print("Only " + str(found['stock']) + " available!")
            return

        free = qty // 3# Buy 3 get 1 free rule
        total = qty * found['price']# Total price for paid items only
        found['stock'] -= (qty + free)# Reduce stock including free items

        # Print a nice-looking receipt for the customer
        print("\n" + "═" * 40)
        print(" WECARE SKINCARE".center(40))
        print("═" * 40)
        print(" Customer:  " + customer_name)
        print(" Product:   " + found['name'])
        print(" Brand:     " + found['brand'])
        print("─" * 40)
        print(" Price:     $" + str(round(found['price'], 2)))
        print(" Quantity:  " + str(qty) + " (+" + str(free) + " free)")
        print("─" * 40)
```

```
        print(" TOTAL:     $" + str(round(total, 2)))
        print("=" * 40)
        print(" Thank you for your purchase! ".center(40))
        print("=" * 40 + "\n")

        # Save products and invoice
        save_products(products)

        # Generate invoice (without date/time)
        invoice_details = "Customer: " + customer_name + "\n"
        invoice_details += "Product: " + found['name'] + "\n"
        invoice_details += "Brand: " + found['brand'] + "\n"
        invoice_details += "Price: $" + str(round(found['price'], 2)) + "\n"
        invoice_details += "Quantity: " + str(qty) + " (+" + str(free) + " free)\n"
        invoice_details += "Total: $" + str(round(total, 2)) + "\n"
        save_invoice(invoice_details)

    except ValueError:
        print("Please enter a valid number") # If input isn't a number, show error

def restock_product(products):# Function to handle adding new stock to a product
    """Handle restocking products."""
    show_products(products) # Show current stock before restocking
    try:
        staff_name = input("Enter staff name: ").strip()
        name = input("Product name: ").strip().lower()
        qty = int(input("Quantity to add: "))
        new_cost = input("New cost (press Enter to keep current): ") # Optional cost
update

        for p in products:# Find the product in the list
```

```
        if p['name'].lower() == name:
            p['stock'] += qty
            if new_cost:
                p['cost'] = float(new_cost) # Update cost if given
                p['price'] = round(p['cost'] * 2, 2)# Recalculate price (cost x 2)
            save_products(products)# Save changes to the file

            # Generate invoice details
            invoice_details = "Staff: " + staff_name + "\n"
            invoice_details += "Product: " + p['name'] + "\n"
            invoice_details += "Brand: " + p['brand'] + "\n"
            invoice_details += "Added Quantity: " + str(qty) + "\n"
            invoice_details += "New Cost: $" + str(round(p['cost'], 2)) + "\n"
            save_invoice(invoice_details)

            # Print confirmation on screen
            print("\n" + "═" * 40)
            print(" RESTOCK CONFIRMATION".center(40))
            print("═" * 40)
            print(" Staff:     " + staff_name)
            print(" Product:   " + p['name'])
            print(" Brand:     " + p['brand'])
            print("─" * 40)
            print(" Added Qty: " + str(qty))
            print(" New Cost:  $" + str(round(p['cost'], 2)))
            print("═" * 40)
            print(" Inventory updated successfully! ".center(40))
            print("═" * 40 + "\n")

            return# Exit after restocking the right product
    print("Product not found!")# If name doesn't match any product
```

```
    except ValueError:
        print("Invalid input!") # Catch errors like typing letters where numbers are expected
```

### 6.3 Read.py

```
# read.py

def read_file(filename="products.txt"):
    """Read product data from a file."""
    products = []# This list will hold all the products we read from the file
    try:
        with open(filename, 'r') as file:# Open the file in read mode
            for line in file: # Go through each line in the file
                if line.strip():# Make sure the line isn't empty
                    # Split the line by commas to get the product details
                    parts = line.strip().split(',')
                    # Extract individual pieces of product info
                    name = parts[0].strip()
                    brand = parts[1].strip()
                    qty = int(parts[2].strip())
                    cost = float(parts[3].strip())
                    country = parts[4].strip()
                    # Create a dictionary for the product
                    product = {
                        'name': name,
                        'brand': brand,
                        'stock': qty,
                        'cost': cost,
                        'country': country,
                        'price': round(cost * 2, 2)
```

```
                }

                products.append(product)# Add the product to the list
        return products # Return the list of all products
    except:# If anything goes wrong (e.g., file not found or data error), show a message
        print("Something went wrong while reading the file.")
        return [] # Return an empty list in case of error
```

### 6.4 Write.py

```
import datetime

def save_products(products, filename="products.txt"):
    """Save products to file."""
    try:
        # Open the file in write mode ('w' means it will overwrite the file)
        with open(filename, 'w') as file:
            # Loop through each product in the list
            for p in products:
                file.write(p['name'] + "," + p['brand'] + "," + str(p['stock']) + "," + str(p['cost']) +
"," + p['country'] + "\n")
    except:
        # If something goes wrong while saving, print an error message
        print("Error saving products file")

def save_invoice(transaction_details, invoice_filename="invoice.txt"):
    """Generate and save invoice to file."""
    try:
        # Open the invoice file in append mode ('a' means it adds to the end of the file)
        with open(invoice_filename, 'a') as file:
            # Write the invoice details followed by two new lines (for spacing)
```

```
        file.write(transaction_details + "\n\n")
    except:
        print("Error saving invoice file")# Print an error message if saving fails
```